Bounding the in-memory block DAG in `f1r3node-rust`

Code analysis of issue #933 and a development plan

Design note for the F1R3FLY.io node team

9 November 2026 — `dev` at `0ccc315d`

1 Summary

The claim is correct and the defect is evident in the code. The in-memory `DagState` in `block-storage/src/rust/dag/block_metadata_store.rs` holds seven collections keyed by block hash or height. Six of them — `dag_set`, `child_map`, `height_map`, `block_number_map`, `main_parent_map` and `self_justification_map` — receive an entry on every insert and have no removal path anywhere in the workspace (outside test fixtures). Only `finalized_block_set` is capped. Finalization advances `last_finalized_block` and sets per-block flags, but never shrinks the indices; and at startup `BlockMetadataStore::try_new` rebuilds all six from every metadata row in LMDB, so memory is proportional to the whole chain history, never to the live frontier. GitHub issue #933 (opened 4 Nov 2026) reports the same structures, with a measured 3,493 bytes of live heap per block and about 37 MB/h of growth per node on an idle 15-node testnet.

The intuition that finalization should allow the in-memory DAG to be "reset" is right in spirit but needs one refinement: the correct shape is a *window* of heights behind the last finalized block (LFB), not a reset to the fringe, because several readers legitimately walk below the LFB. The repository already contains the machinery to make this safe — the "restore horizon" contract for nodes joined from a Last Finalized State (LFS) snapshot, under which a block the node does not hold is reported as `BlockNotHeld` and deferred rather than treated as a fault. A windowed DAG makes every node look like an LFS-restored node whose horizon advances with finality. The plan below reuses that contract rather than inventing a second one.

2 What the code does today

2.1 The growing structures

```
// block_metadata_store.rs
pub(crate) struct DagState {
    pub(crate) dag_set: imbl::HashSet<BlockHash>,
    pub(crate) child_map: imbl::HashMap<BlockHash, imbl::HashSet<BlockHash>>,
    pub(crate) height_map: imbl::OrdMap<i64, imbl::HashSet<BlockHash>>,
    pub(crate) block_number_map: imbl::HashMap<BlockHash, i64>,
    pub(crate) main_parent_map: imbl::HashMap<BlockHash, BlockHash>,
    pub(crate) self_justification_map: imbl::HashMap<BlockHash, BlockHash>,
    pub(crate) last_finalized_block: Option<(BlockHash, i64)>,
    pub(crate) finalized_block_set: imbl::HashSet<BlockHash>,
}
```

`add_block_to_dag_state` inserts into all of the first six on every `add`. The only remover in the file is `prune_finalized_cache_if_needed`, which trims `finalized_block_set` from 50,000 to 25,000 entries (and does so in arbitrary iteration order, not oldest-first; `is_finalized` falls back to the LMDB metadata row, so this is a cache miss rather than a correctness bug). A workspace-wide search for `.remove(` on the other six fields finds nothing outside tests.

The design is inherited from the Scala `BlockMetadataStore` the file header cites, which also kept its DAG indices in memory for the life of the process; the Rust port adds three more per-block maps (`block_number_map`, `main_parent_map`, `self_justification_map`) as hot-path caches, roughly doubling the per-block footprint. The measured ∼3.5 KB per block is consistent with six `imbl` HAMT entries holding 32-byte `Bytes` keys plus a freshly allocated `imbl::HashSet` per block in `child_map`; the child sets are the likely dominant term and the profile in Phase 0 should confirm that.

2.2 What finalization does, and does not, do

`BlockDagKeyValueStorage::record_directly_finalized` (called from `finalization_runner.rs`) walks the held, unfinalized ancestry of the new LFB, runs the finalization effect, then `record_finalized` marks the blocks finalized in LMDB, adds them to `finalized_block_set` and advances `last_finalized_block`. Nothing is removed from the six indices. `propagate_ft_to_finalized_blocks` then scans every finalized block whose cached fault tolerance is below the new value — a second place where cost grows with history rather than with the frontier (partly mitigated by the `ft_lower_bound` skip).

2.3 Startup

`BlockMetadataStore::try_new` calls `store.collect` over the entire `block-metadata` LMDB table and folds every row into a fresh `DagState`. Both the time and the resident memory of a restart therefore scale with total chain length. Issue #923 (disk growth) is the on-disk twin; this note deliberately leaves LMDB contents intact and bounds only the in-memory view.

3 Why a reset to the fringe is not quite right

A `KeyValueDagRepresentation` snapshot answers two different kinds of question: *membership* ("does this node hold block h?") and *structure* (children, heights, main-parent spine). Several production readers ask these questions about blocks *below* the LFB:

Late blocks from lagging validators.
The parent-depth rule (`validate.rs`, "ANCHOR: the block's OWN parent frontier, never this node's tip") deliberately allows a block whose parents sit arbitrarily far below the validating node's LFB. Validation resolves those parents through `dag.lookup`, which is gated on `dag_set.contains`. If the parent were evicted from `dag_set` while still present in LMDB, `lookup` would return `None`, the dependency gate would park the block in the Casper buffer waiting for a block the node already has, and nothing would ever release it — the stall class of #690 and the #306 "not held" storm, self-inflicted. **Membership must stay truthful for every block the node holds, pruned or not.**

Mergeable-channel GC.
`mergeable_channels_gc.rs` enumerates `height_map` from its last swept height up to `floor.block_number - (max_parent_depth + mergeable_channels_gc_depth_buffer)`

and needs `children` for each candidate. If heights are evicted before the sweep has enumerated them, their mergeable data is never collected. The retention depth below the floor must cover this ceiling, and eviction must be ordered after enumeration (or the sweep's `swept_height` must be persisted so a restart does not lose it; today it is process-local and restarts from genesis).

Repeat-deploy scope walks.
`walk_blocks_in_scope` (`proto_util.rs`, #776) descends `deploy_lifespan` heights below the *proposing block's parents*, reading `dag.block_number` from the in-memory index to prune the walk. With the own-frontier parent rule those parents can be below the node's LFB.

Silent validators.
`latest_messages()`, `non_finalized_blocks()` and the clique oracle follow a validator's latest message and self-justification chain; a validator silent for longer than the window has a latest message below it.

Block API, history horizon, replay.
`topo_sort`, `get_max_height`, `find` (truncated-hash search over `dag_set`), `descendants` (legacy merge path), `rspace_history_horizon.rs`, the replay in `initializing.rs` and `BlockStateManager::is_empty` (`topo_sort(0, Some(1))`) all assume the height index covers the whole chain. `genesis_hash()` falls back to `height_map.get(&0)` when the genesis register is unset; the register is written on every bootstrap path in `initializing.rs`, so the fallback can go.

Fork choice itself (`estimator.rs`) starts at the LFB and walks *children* upward, and the DAG merger scopes blocks between the LFB and the tips; both live entirely in the retained window and need no change.

4 Design

4.1 Principle

Keep LMDB complete. Make the in-memory indices a cache over a height window $[W, \infty)$ where W is the *window floor*, and make every accessor on `KeyValueDagRepresentation` fall through to the LMDB metadata row for a block not in the window. Everything a walk needs about an old block — `block_number`, `parents[0]` (main parent), the sender's own justification, the `finalized` flag — is already in `BlockMetadata`. The one relation that is *not* derivable from a metadata row is *children*; the design persists a height index to LMDB and makes children-below-window either a persisted index or an explicitly unsupported query, depending on the caller.

4.2 Window floor

After each successful `record_directly_finalized`,

$$W = \text{LFB.block_number} - D,$$
$$D = \max\big(\texttt{max_parent_depth} + \texttt{mergeable_channels_gc_depth_buffer},\ \texttt{deploy_lifespan},\ \texttt{dag_memory_window_min}\big),$$

with `dag-memory-window-min` a new `casper` configuration key (suggested default 1,000 heights). W is monotone non-decreasing and is additionally capped at the GC sweep's enumerated height (Phase 3). With the defaults in `defaults.conf` (`max-parent-depth = 15`, `deploy-lifespan = 50`, buffer 10) the minimum dominates, and at the observed 5–9 blocks per height a 1,000-height window costs roughly 25–30 MB of index memory — bounded, and tunable by operators.

4.3 Accessor semantics after the change

Accessor	In window	Below window (block_number $< W$)
`contains` / `lookup`	in-memory set, then LMDB row	LMDB row (truthful membership)
`block_number`, `main_parent`, `self_justification`	in-memory map	derived from LMDB row
`is_finalized`	set, then LMDB	LMDB (unchanged)
`children`	in-memory map	LMDB `children-index`, or `None` (see 4.4)
`topo_sort`, `get_max_height`	in-memory `height_map`	LMDB `height-index` range scan
`find` (truncated hash)	in-memory scan	LMDB prefix scan on metadata keys
`genesis_hash`	register	register (drop the height-0 fallback)

Snapshot semantics. A representation is a point-in-time snapshot; falling through to LMDB could reveal a block inserted *after* the snapshot. Gate the fall-through on the row's `block_number` being below the snapshot's own W (captured at `get_representation` time). The residual leak is a late block from a lagging validator that lands below W between a snapshot and its use; such a block is settled-history territory and the only consumers that could observe it are read-only API calls. Record this as an accepted deviation in the module docs.

4.4 Children below the window

Production callers of `children`: the estimator (walks up from the LFB, always in window), `mergeable_channels_gc::reaches_ancestor_check` (candidates are at most D below the floor, in window by construction of D), and `descendants` (legacy merge path, base is at or above the LFB). No production caller needs children of a block below W. Phase 1 therefore returns `None` below the window and documents it; a persisted children index is listed as an optional follow-up only if the block API's graph renderers need it.

4.5 Eviction

Eviction is by height, under `global_lock.write()`, immediately after `record_finalized`: for each height h in `height_map.range(..W)`, remove its hashes from the six maps and remove the height key. Eviction ignores finalization status — an unfinalized orphan below W stays in LMDB and remains reachable through fall-through, and if a later LFB's ancestry merges it, `held_ancestors` still finds and finalizes it. The GC sweep keeps its own `pending` set, so evicting orphans from the DAG indices does not lose them for GC. Because `imbl` collections are persistent, eviction is $O(\text{evicted} \cdot \log n)$ and outstanding snapshots are unaffected.

5 Development plan

Each phase is independently mergeable and behaviour-preserving until Phase 3 turns eviction on behind a configuration flag.

Phase 0 — Reproduce and instrument (1–2 days)

1. Land the counting-allocator measurement from #933 as `block-storage/tests/dag_state_memory_bound.rs`: 120,000 shard-shaped inserts,

assert bytes/block over the last 100,000; record the 3.5 KB baseline and leave the assertion as a regression bound once Phase 3 lands.
2. Add gauges `dag.window_floor` and `dag.evicted_total` alongside the existing `DAG_BLOCKS_SIZE_METRIC` / `DAG_CHILDREN_INDEX_SIZE_METRIC` / `DAG_HEIGHTS_SIZE_METRIC` in `multi_parent_casper/snapshot.rs`; these are the soak acceptance signals.
3. Confirm with a heap profile which of the six collections dominates.

Phase 1 — Truthful absence: fall-through accessors (3–5 days)

1. Add `window_floor: i64` to `DagState` and to `KeyValueDagRepresentation` (initially `i64::MIN`, so nothing changes).
2. Rewrite `lookup`, `contains`, `block_number`, `main_parent`, `self_justification` and their `_metered` / `_unsafe` variants to fall through to `block_metadata_index.get` when the hash is not in `dag_set`, gated as in 4.3. Reuse `block_metadata_to_info` for the derivations.
3. Replace the three production direct-field reads with accessors: `clique_oracle.rs:250` (`block_number_map`), `mergeable_channels_gc.rs:258` (`height_map`, via a new `heights_in_range` accessor), and the size gauges in `snapshot.rs`. Then narrow the field visibility so the compiler enforces it (tests use the existing `from_parts` escape hatch).
4. Drop the `height_map.get(&0)` fallback in `genesis_hash`; change `BlockStateManager::is_empty` to test the genesis register or `dag_set.is_empty()`.
5. Make `validate_dag_state` compute contiguity from W rather than from the minimum height, so eviction never trips the warning.
6. All existing tests must pass unchanged; add unit tests for each accessor with a block present in LMDB but absent from `dag_set`.

Phase 2 — Persisted height index and bounded startup (3–4 days)

1. Add an LMDB store `height-index` (`i64` $\to$ set of hashes), written in `insert_internal` under `global_lock`, with a one-time backfill from the metadata scan when the store is empty at startup (same shape as the carrier-index watermark bootstrap).
2. Route `topo_sort` and `get_max_height` through the in-memory map when the range is inside the window and through the LMDB index otherwise; `find` falls back to a key-prefix scan on `block-metadata`.
3. Change `try_new` to rebuild only heights $\geq W_0$, where W_0 is derived from the persisted LFB's block number and D; LMDB remains the source of truth for everything below. Startup becomes $O(\text{window})$.

Phase 3 — Eviction behind a flag (3–5 days)

1. Add `casper.dag-memory-window-min` and `casper.dag-memory-window-enabled` (default off for one release).
2. Implement `evict_below(W)` in `BlockMetadataStore` and call it from `record_directly_finalized`'s persist closure after `record_finalized`, computing W as in 4.2.
3. Couple to GC: persist `GcSweep::swept_height` in an LMDB meta slot and cap W at it, or move enumeration to the persisted height index so the sweep no longer depends on in-memory `height_map`. The second is cleaner and also fixes the restart-from-genesis re-enumeration.
4. Let `finalized_block_set` eviction follow W instead of the arbitrary 50k/25k trim.

Phase 4 — Verification (soak, 1 week elapsed)

- **Unit/property.** Property test: for a random DAG and random W, every accessor returns the same answer with and without eviction (fall-through equivalence). Add to `casper/tests/fork_choice/`.
- **Lagging validator.** Integration test in `casper/tests/batch1/`: a validator silent for $> D$ heights proposes a block citing parents below W; the block must be admitted, not parked in the Casper buffer.
- **GC.** Mergeable data of blocks evicted from memory is still collected; sweep survives a restart.
- **LFS joiner and restart.** A node restored from an anchor and a node restarted with a populated database both come up with a bounded `DAG_BLOCKS_SIZE` and serve `topo_sort` over pruned heights.
- **Soak.** Repeat the #933 idle testnet: 20 h, 10 validators, 5 observers. Acceptance: container anon memory flat after warm-up (< 2 MB/h drift), `dag_blocks_size` bounded by $D\times$ blocks-per-height, finalization p95 not worse than `dev`.
- Enable the flag by default once the soak passes; remove the flag one release later.

6 Risks and open decisions

- **Snapshot leak (4.3).** Accepted deviation; document it. If a reviewer objects, the alternative is to stamp each metadata row with the `dag_generation` at insert and gate fall-through on it, at the cost of a metadata schema bump.
- **Fall-through cost.** Each below-window accessor call is one LMDB point read instead of a map lookup. Only lagging-validator validation and API calls hit it; the hot paths (fork choice, merge, finalizer) stay in window. Confirm with the `dag.insert.time` and finalizer timing histograms during the soak.
- **Adjacent unbounded state, out of scope here.** `invalid_blocks_index` and `latest_messages_index` are materialised with `to_map()` on every `get_representation`; the equivocation tracker grows with equivocations; `propagate_ft_to_finalized_blocks` scans all finalized blocks. None grows per block at idle, but each deserves its own ticket.
- **Disk (#923).** This plan keeps LMDB complete by design. Pruning LMDB below an archival horizon is a separate decision with network-protocol implications (what a node can serve to joiners) and should not be bundled with the memory fix.